



CLASSIFICATION: OPERATOR EYES ONLY

VOLUME I

ENYGMA

Operator's Manual

A complete reference for building, deploying and operating ENYGMA: a private meeting-intelligence system for a single operator, running on hardware they own. Covers the host, the pipeline, authentication, recovery and daily use, with the reasoning behind every decision that would otherwise look arbitrary.

AUTHOR

ARKHM

OPERATOR

YAHYA

EDITION

MK II.23

DATE

2026-09-02

Colophon

FRONT MATTER

MK II.23 / 2026-09-02

This manual documents **ENYGMA**: a meeting-intelligence system built for one person, on one machine, with no account to sign up for and no company between the operator and their own recordings. It is written for a technically-literate operator with shell access to the host.

Every value, command and design decision here is documented with the reason it was made, so that the system can be rebuilt from a clean machine by someone with no access to a language model and no memory of the original build.

Edition

AUTHOR	ARKHM
VOLUME	I, Operation
EDITION	Mk II.23 2026-09-02
HOST	spark-4d80 , Ubuntu, Python 3.12
ADDRESS	https://enygma.arkhm.io
REPOSITORY	github.com/dshafique/ENYGMA
MANUAL SOURCE	docs/manual/ENYGMA-OPERATORS-MANUAL.html

Scope of this edition

This edition is the operator's reference for the running system. It covers everything needed to rebuild it, everything needed to run it day to day, and the failure modes that have actually occurred rather than the ones that theoretically might.

What this edition documents in full:

- **Foreword** | what ENYGMA is, the five tenets, how to read this.
- **Concept** | the product surface, the architecture, the pipeline end to end.
- **Bill of Materials** | host, software, accounts, network, cost.
- **Build** | deployment, configuration, first run, verification.
- **Access & Auth** | Cloudflare Tunnel, passkeys, setup codes, PIN, recovery, PWA.

- **Operating** | daily use, backups and restore, failure modes, releases.

What this edition does not cover:

- PHNTM. The two systems share a machine and nothing else. Where this manual mentions PHNTM it is only to describe what must not be disturbed.
- The HiNotes recorder integration, which exists in the source but is gated off. Audio arrives by drag and drop.
- Verification of transcript accuracy against a known-good reference. That work is outstanding and is named as such in Chapter 18.

Conventions used in this manual

CONVENTION	MEANING
<code>monospace</code>	A literal: a command, a path, a file name, an environment variable, a value to type exactly.
Dark terminal block	A command to run, with its real output. Output shown is output actually observed, not illustrative.
Light code block	File contents, or code quoted from the source tree.
Note	Something that will save time or prevent confusion.
Warning	Something that can lose data, break access, or disturb PHNTM.
Scar	A decision that looks strange until you know the bug behind it. These are the most useful paragraphs in the manual.

NOTE

Commands and outputs in this manual were run against the working system at the edition's date. Where the manual and the source tree disagree, the source is canonical and the manual should be re-issued.

Versioning

The manual's edition number is the application's mark: **Mk II.N**, where N is the commit count of the repository the build came from. A manual marked Mk II.5 describes the system as it stood at commit 5. The application reports its own mark in the top right of every screen and at `/healthz`, so a manual and a running system can always be checked against each other in one glance.

Table of Contents

FRONT MATTER

MK II.23 / 2026-09-02

PART 0 | FOREWORD

- 0.1 What ENYGMA is
- 0.2 The Five Tenets
- 0.3 How to read this manual

PART 1 | CONCEPT

- 1 The product surface
- 2 Architecture overview
- 3 Data flow, end to end

PART 2 | BILL OF MATERIALS

- 4 The host
- 5 Software stack
- 6 Accounts, network topology and cost

PART 3 | BUILD

- 7 Deployment
- 8 Configuration
- 9 First run and verification

PART 4 | ACCESS & AUTH

- 10 Cloudflare Tunnel and DNS
- 11 Passkeys and the freshness window
- 12 Setup codes, PIN and adding a device
- 13 Recovery procedures and threat model

14 Installing as an app

PART 5 | OPERATING

15 Daily use

16 Backups and restore

17 Failure modes

18 Releases, and what is still unproven

PART 6 | WHAT IT WRITES

19 The Friday note

20 Documents

21 How ENYGMA writes

22 The native shell

APPENDICES

A Environment variables

B HTTP surface

C Source tree map

D The scar list

Foreword

What ENYGMA is, the five commitments it is built on, and how to read the rest of this.

-
- 0.1 What ENYGMA is
 - 0.2 The Five Tenets
 - 0.3 How to read this manual

0.1 | What ENYGMA is

PART 0 • FOREWORD

0.1

ENYGMA turns a recording of a meeting into something you can act on: an attributed transcript, a summary where every claim points back at the moment it was said, and a list of what anybody committed to do.

The problem it solves

An intern sits in meetings where decisions are made quickly, by people who already share years of context, using words nobody stops to define. Two hours later he has three pages of notes, no idea which parts mattered, and a handful of terms he did not want to interrupt to ask about.

Recording the meeting does not fix this. A ninety-minute audio file is worse than notes: it contains everything and surfaces nothing. What is needed is the same recording, indexed by speaker and by moment, with the decisions pulled out and every one of them still attached to the audio that proves it.

What makes it different from a transcription service

Three things, all of which follow from who it is for.

It runs on hardware its operator owns. There is no account, no dashboard belonging to somebody else, and no company that can change its terms, raise its price, or read the recordings. The only third party that ever sees the audio is the transcription model, and that relationship is one API key that the operator holds and can revoke.

Every claim is checkable. A summary that asserts a decision without pointing at the moment it was made is an assertion the operator cannot verify. Every decision, open question and action in ENYGMA carries a timestamp that seeks the audio. If a claim cannot point at a moment, it is left out.

It teaches. Hold any word in a transcript and ENYGMA explains it; ask for more and the term opens a conversation. The system is built on the assumption that its operator is learning, and that the words he did not want to interrupt to ask about are the highest-value thing in the recording.

What it is not

- **Not multi-tenant**, and deliberately not built so that it could become multi-tenant later. One operator, one database, one unix user. Every place where a second user would need to be accommodated is a place where a simpler decision was made instead.
- **Not a note-taking app**. It does not compete with the operator's own notes; it indexes the recording those notes were taken from.
- **Not a system of record for anyone else**. Nothing synchronises outward. A meeting can be copied out as Markdown on demand, which is a one-directional escape hatch, not an integration.
- **Not a source of truth about people**. Speaker names are assigned by the operator, by hand, and the model never guesses one. Two similar names are two people until the operator says otherwise.

NOTE

ENYGMA shares a machine with PHNTM and shares nothing else: not a database, not a session secret, not an API key, not a unix user. The isolation is enforced by file permissions, and Chapter 4 shows the proof rather than asserting it.

0.2 | The Five Tenets

PART 0 • FOREWORD

0.2

Five commitments. Every design decision in this system can be traced to one of them, and where two of them conflict the earlier one wins.

TENET 01

No password exists in this product

Authentication is a passkey: a private key held by the device's secure hardware, released by a fingerprint. There is nothing to phish, nothing to reuse from another site, and nothing a database breach could reveal, because the server stores only public keys.

A PIN and recovery codes exist as the way back in when the passkey is unavailable. They are hashed with Argon2id, never a fast hash, and they sit behind a single lockout counter shared by every method — five failures across passkey and PIN combined, not five each.

TENET 02

Isolation is filesystem permissions, not discipline

ENYGMA runs as its own unix user, with its own home directory at mode 750 and its data and audio directories at 700. The isolation from PHNTM is not a convention that a future change might forget; it is a property the kernel enforces on every read.

The session cookie is named with the `__Host-` prefix, which forbids a `Domain` attribute. The cookie therefore *cannot* be scoped to `arkhm.io` — not by policy, but by construction.

TENET 03

Migrations are forever-additive

Never drop a column, never rename one. To remove a column, stop writing to it. A migration that only ever adds is a migration that can be applied to any older database, which means a backup from any point in the system's life can still be restored and brought forward.

TENET 04

Every claim carries the moment it came from

A summary is a set of assertions about what happened. An assertion the operator cannot check is worse than no assertion, because it is trusted.

Every decision, open question and action in ENYGMA is stored with the millisecond offset it came from, rendered as a timestamp the operator can click to hear it. The prompt sent to the model says explicitly: if you cannot point at a moment, leave the item out. This survives the copy-out too — a meeting exported as Markdown carries its timestamps with it, because pasting a summary somewhere else is exactly where an unsourced claim does damage.

TENET 05

Empty beats invented

Where the system has no real answer it says so and shows nothing. Home has a panel for headlines in the operator's field; no feed is connected, so it says that plainly rather than displaying something plausible. Chat, with no model reachable, returns what the glossary actually knows and states that it cannot go further. The transcription prompt is told to write `[inaudible]` rather than invent words.

A blank panel is a small failure. A confidently wrong one is a large failure that looks like a success, and the operator has no way to tell them apart.

0.3 | How to read this manual

The two reading orders

Reader A, the operator

You want to use the system that is already running. Read Chapter 1 for what the five tabs are, Chapter 15 for daily use, and Chapter 12 for how to get it onto a second device. Keep Chapter 17 nearby: it is the list of things that can go wrong and what each one means. Everything else can wait until it breaks.

Reader B, the builder

You are rebuilding this from a clean machine, or you are the person who inherits it. Read straight through. Parts 2 and 3 are the build; Part 4 is the half that is easy to get subtly wrong and hard to debug afterwards. Appendix D — the scar list — will save you the most time of anything here.

What you need to know going in

- A shell, `ssh`, and enough comfort with a Linux host to read `journalctl` output without panic.
- Nothing about Python. You will run scripts, not write them.
- Nothing about the browser. The interface explains itself.

If something here is wrong

The source tree is canonical. If a command in this manual fails and the source disagrees with the manual, the source is right and this document is stale — re-issue it at the next edition rather than working around it. The manual's edition is the commit it was written against, so the disagreement can always be located precisely.

SCAR

An earlier deployment of this system re-installed a stale copy of itself and reported complete success: correct output, all tests passing, healthy service. It was three days behind. Every version number in this manual, and the mark in the corner of every screen, exists because of that afternoon.

Concept

What the operator sees, what runs underneath it, and the path a recording takes from a dropped file to a checkable summary.

-
- 01 The product surface
 - 02 Architecture overview
 - 03 Data flow, end to end

Chapter 1 | The product surface

Six destinations, one shell. A nav rail on a desktop, a tab bar on a handset, and the same six places either way.

TAB	ANSWERS	HOLDS
Home	What should I look at now?	The week's note when there is one, then recent meetings, open actions, headlines, what ENYGMA has learned, counts. Every panel is a door into another tab, except the note, which is the one thing here that leaves the building (Chapter 19).
Meetings	What happened in this one?	The list and the reader: search, status filters, day groups; then speakers, summary, transcript, actions and the audio.
Chat	What does this mean?	Threads, with memory: the last twelve turns go into every answer, so a follow-up has something to follow. A thread can start empty or from a word held down in a transcript. Ask for a file here and you get a file (Chapter 20).
Action items	What do I owe anybody?	Every commitment across every meeting, open and done, each linking back to the moment it was said.
Directory	Who is this person?	Everyone who has spoken, and the meetings they appeared in. A person exists because a speaker label was given a name.
Library	What does ENYGMA know that was already true?	Documents and notes. A meeting records something that happened; a document is context that existed before it. Everything here is searched when a question is asked in Chat.

Settings is not a tab. It sits at the foot of the rail on a desktop and in the header row on a handset, because it is a place you visit rarely and deliberately.

Meetings without a recording

Sometimes nobody recorded it and all that exists is somebody else's minutes, or a meeting tool's own summary. Drop that text file on the same dropzone and it becomes a meeting: title, date and attendees read out of the notes, an abstract, decisions, open questions and actions.

It is marked **FROM NOTES** everywhere it appears, and it carries **no timestamps at all**. There is no recording for a claim to point at, and a timestamp on a meeting nobody

recorded would be a fabrication dressed as evidence. The reader says so in a banner, the Transcript tab is called Notes, and the Markdown copy carries the same warning out with it.

Attendees named in the notes are recorded as having *attended*, with zero turns, because being in the room is not the same claim as having spoken.

NOTE

Notes about a meeting go here. Reference material — a spec, a runbook, a paper — goes in the Library, which is context that was already true rather than a record of an event. A file too short to be a meeting is refused with a sentence pointing at the Library.

What it accepts

Audio, dropped onto the Meetings tab: `mp3`, `wav`, `m4a`, `aac`, `ogg`, `flac`, `aiff`. Up to 500 MB each by default. Files are content addressed by SHA-256, so the same recording dropped twice is one meeting whatever it was called the second time. That is the only deduplication rule: exact, never fuzzy, never guessing that two files are "probably" the same.

Action items have four states

A checkbox can only say finished or not finished, which makes everything consciously turned down look identical to everything nobody has got to yet. A list that cannot tell those apart stops being read.

STATE	MEANS
Open	Owed, and nothing has happened yet. The default.
Pending	Waiting on somebody or something else. Not his move.
Done	Finished.
Rejected	Turned down on purpose. It stays on the list, struck through, with the reason if one was given.

Only **open** counts as something owed: it is what Home shows and what the count in the meeting list means. Rejected items are kept rather than deleted, because "we decided not to" is a fact worth being able to find later.

What it produces

1. The attributed transcript

Every segment carries a speaker label, a start and an end. Labels begin as `SPEAKER 1`, `SPEAKER 2` and are given real names by the operator, by hand. The model is instructed never to guess a name even if one is said aloud, because a name in a transcript is evidence and a guessed name is a fabrication that looks like evidence.

2. The checkable summary

An abstract of two or three sentences, then decisions, open questions and actions — each one carrying the timestamp it came from. Clicking a timestamp seeks the audio to that moment.

3. The glossary and the conversation

Holding a word in the transcript raises a small panel with a definition and a *Learn more* that opens a Chat thread named for that term, with the question already asked and answered. The glossary ships with the vocabulary of the operator's field and grows by use.

4. Answers drawn on his own material

The Library holds documents and notes: a spec, a runbook, an onboarding note, a paper somebody sent. Every question asked in Chat searches it first, and the answer names which documents it drew on so a claim can be traced back.

Retrieval is SQLite's own full-text index with stemming, not vector embeddings. For one person with tens of documents that finds the right passage, costs nothing per query, needs no API key, keeps working when the model is unreachable, and can be reasoned about when it returns the wrong thing. A passage has to share more than one distinctive word with the question, or one word that is rare across the whole library — otherwise nothing is returned at all, because a coincidence presented as a source is the confident wrongness this system exists to refuse.

5. A copy, on demand

Copy as Markdown puts the whole meeting on the clipboard: title, date, who was present, abstract, decisions and questions with their timestamps, actions as checkboxes with owners resolved to real names, and a closing line stating which model produced it and that speaker names are not model-verified. One direction, on demand, nothing stored.

Chapter 2 | Architecture overview

PART 1 · CONCEPT

02

Component map



Process model

One process. Uvicorn serves the HTTP surface on the main thread; a single background thread runs the transcription worker, one recording at a time. There is no queue broker, no second service and no scheduler, because one operator dropping a few files a week does not have a queueing problem, and machinery that is not there cannot fail at three in the morning.

Why FastAPI

Server-rendered Jinja templates, no front-end framework, no build step. The entire client is one CSS file and one JavaScript module, both served directly. A deploy is copying files and restarting a service. There is no `node_modules` anywhere in the runtime path, which means there is no supply chain in the runtime path either.

Why SQLite

One operator, one machine, single-digit gigabytes. SQLite in WAL mode handles this with room to spare, and it makes the backup story trivial: the entire system's memory is one file that can be copied, verified and restored in seconds. A database server would add a second thing to secure, a second thing to back up, and a second thing to be running before the app can start.

NOTE

WAL mode is why backups are taken through SQLite's own backup API rather than `cp`. A file copy taken mid-write is a torn database that looks fine until the day you need it. See Chapter 16.

Chapter 3 | Data flow, end to end

PART 1 • CONCEPT

03

Nine stages from a dropped file to a summary on screen. Every stage either advances the recording's status or fails it with a sentence.

The pipeline

1. **Drop.** The browser posts the file to `/upload`. A stale session is refused here with `reason: stale`, which raises the re-auth sheet rather than failing silently.
2. **Validate before writing a byte.** Extension must be a known audio type; size must be under the ceiling. Rejection is a sentence saying which rule was broken.
3. **Stream and hash together.** The file is written to a temporary name while SHA-256 is computed over the same pass, so a 400 MB file is read once, not twice.
4. **Deduplicate.** If that hash already exists, the temporary file is discarded and the existing recording is returned, marked as a duplicate. Nothing is overwritten.
5. **Commit.** The file is renamed to `uploads/<first-16-of-hash>.<ext>` and a row is inserted with status `queued`. The path is stored POSIX-style regardless of the client's platform.
6. **Claim.** The worker takes one queued recording inside a `BEGIN IMMEDIATE` transaction and marks it `transcribing`, so two claims can never race.
7. **Window, if it is long.** Speaker attribution is only supported up to thirty minutes of audio. Anything longer is cut into overlapping windows with `ffmpeg` and each is transcribed on its own; see Chapter 17.
8. **Transcribe.** Under 20 MB the audio goes inline as base64; above that it is uploaded via the Files API and referenced by URI. The model returns JSON segments with speaker, start, end and text.
9. **Summarise.** The transcript is rendered to timestamped text and sent back for an abstract, decisions, questions and actions — each with the moment it came from.
10. **Write results atomically.** Segments, speakers, summary and actions are written in one transaction, and the recording flips to `ready`. Completed actions are preserved across a re-run; only open ones are replaced.

Status states, complete list

STATE	MEANS	WHAT MOVES IT ON
● <code>queued</code>	Stored on disk, waiting for the worker.	The worker, within the poll interval — three seconds by default.
● <code>transcribing</code>	Claimed. The model is working.	Success, or an exception that fails it with a reason.
● <code>ready</code>	Transcript, summary and actions are written.	Nothing. This is the resting state.
● <code>failed</code>	Something threw. The reason is stored and shown verbatim.	<i>Try again</i> on the meeting, which requeues it without re-uploading.

SCAR

A recording is marked `transcribing` *before* the work starts. If the process dies in that gap — a restart, the memory ceiling, a power cut — nothing would ever pick it up again, because the worker only claims `queued`. The operator would watch a progress bar that could never finish. `requeue_stuck()` runs at startup and puts them back, with a note saying so.

The stub backend

With `ENYGMA_PIPELINE=stub` the entire application runs with no API key and no network, producing deterministic placeholder transcripts. This is not a toy: it is how the interface was built before the bill started, it is what the test suite runs against so the tests never depend on a third party being up, and it is the safe default in a fresh `.env`.

The interface never lets the stub be mistaken for the real thing. An amber banner sits above the dropzone, and Settings reports the pipeline as `STUB`.

Bill of Materials

The machine, the software on it, the accounts it depends on,
and what the whole thing costs to run.

-
- 04 The host
 - 05 Software stack
 - 06 Accounts, network topology and cost

Chapter 4 | The host

PART 2 • BILL OF MATERIALS

04

Reference specification

HOSTNAME	spark-4d80
OS	Ubuntu, systemd
PYTHON	3.12 (3.10 is the floor; the installer refuses below it)
DISK	3.7 TB NVMe, shared with PHNTM
MEMORY CEILING	8 GB, enforced by systemd <code>MemoryMax</code>
PORT	4073, loopback only. PHNTM holds 4063.
UNIX USER	enygma, home <code>/home/enygma</code> , app in <code>app/</code>

Why a memory ceiling

The machine is shared. A leak or a runaway on ENYGMA's side must not be able to pressure PHNTM out of memory. `MemoryMax=8G` means the kernel kills ENYGMA rather than letting it take the machine down with it, and `Restart=on-failure` brings it back. The interrupted-recording recovery in Chapter 3 exists precisely so that this is survivable.

Isolation, proved rather than asserted

The installer sets the home directory to 750 and the data and upload directories to 700, then verifies it by attempting the read as another account. The relevant part of the install output:

```
$ sudo bash deploy/install-on-spark.sh
=> checks
  healthz: {"ok":true,"app":"enygma","port":4073,"version":"7.2026.08.27.0553"}
  no other account can read the database, the audio or the secret
```

SCAR

`adduser` creates home directories world-readable on most distributions. The first version of the installer used it and stopped there, which meant the `arkhm` account could read every one of Yahya's recordings and his database. This was found by running the installer as root in a scratch container and actually trying the read. The `chmod` lines and the proof loop both date from that.

Disk

Audio costs roughly 75 MB per hour of meeting. The alarm is set at 200 GB, which is about 2,600 hours. Quotas were preferred but are not enabled on this filesystem, so an hourly alarm is installed instead — `/etc/cron.hourly/enygma-disk`, reporting to `journalctl -t enygma-disk`.

WARNING

An alarm reports; it does not stop the write. SQLite behaves badly on a full disk, and the disk is shared with PHNTM. If the alarm fires, act on it.

Chapter 5 | Software stack

PART 2 • BILL OF MATERIALS

05

Python dependencies

PACKAGE	PINNED	WHY THIS ONE
<code>fastapi</code>	0.115.6	Routing, dependency injection, and typed request handling without a framework's worth of ceremony.
<code>uvicorn[standard]</code>	0.34.0	The ASGI server. <code>standard</code> brings the C-accelerated event loop and HTTP parser.
<code>jinja2</code>	3.1.5	Server-rendered templates. No build step, no hydration, no framework.
<code>python-multipart</code>	0.0.20	Required for file uploads. Without it the dropzone posts into a void.
<code>itsdangerous</code>	2.2.0	Signs the session cookie with a timestamp, which is what makes the freshness window possible.
<code>python-docx</code>	1.1.2	The four document renderers. Every one is a pip wheel with no system package behind it, which is what keeps the install a two minute job: a renderer wanting LibreOffice or a headless browser would not survive <code>install-on-spark.sh</code> on a machine with no display. The PDF is built directly rather than printed from a browser for exactly that reason.
<code>python-pptx</code>	1.0.2	
<code>openpyxl</code>	3.1.5	
<code>reportlab</code>	4.2.5	
<code>webauthn</code>	2.5.0	Passkey registration and assertion verification. Server-side challenge handling.
<code>argon2-cffi</code>	23.1.0	Argon2id for the PIN, setup codes and recovery codes. Deliberately slow.
<code>httpx</code>	0.28.1	HTTP client, and the test client's transport.
<code>google-genai</code>	2.20.0	The Gemini backend. Transcription and summarisation.
<code>pytest</code>	8.3.4	90 tests. The installer runs them before it restarts the service.

SCAR

`google-genai` was missing from this list for the whole of the build. The application ran, the tests passed, and `ENYGMA_PIPELINE=gemini` had never once been capable of working — the first real meeting failed on `ImportError`. There is now a test asserting the dependency is declared, and the backend's import error names the command to fix it instead of saying *cannot import name 'genai'*.

Front-end stack

There isn't one. Two CSS files and one JavaScript module, served directly, no bundler.

- `tokens.css` is *generated* by `tools/gen_tokens.py` from a single palette. Raw hex anywhere else fails review.
- `app.css` contains references only — every colour is a `var()`.
- `app.js` is one ES module. Every block guards on its own elements, so a page without a dropzone simply skips that block.

SCAR

An undefined CSS custom property does not warn. The whole declaration becomes invalid, the browser drops it, and a layout quietly collapses with nothing in the console.

`tools/check_tokens.py` fails the build on any `var()` that nothing defines, and the installer runs it. It has caught this class of bug more than once.

Fonts and the mark

Inter for prose, JetBrains Mono for labels, timestamps and anything that is a literal. The mark is a 4×4 cipher grid on a 64-unit box — cell 13, gap 3, offset 1.5, radius 1.6, with four cells voided — and it is drawn from those numbers everywhere it appears, including the PWA launcher icons, which are generated by `tools/gen_icons.py` rather than traced. The icon and the mark cannot drift apart because they are the same specification.

Chapter 6 | Accounts, network topology and cost

PART 2 • BILL OF MATERIALS

06

Required accounts

Cloudflare — free

Holds the `arkhm.io` zone and terminates the tunnel. Already in use for PHNTM; ENYGMA adds one hostname and one DNS record to the existing account and tunnel.

Google Cloud / AI Studio — paid tier, required

Supplies the Gemini API key. **The paid tier is not optional here, and not for rate limits.** On the free tier Google's terms permit prompts and responses to be used to improve their products. What ENYGMA sends is other people's voices, client names and contract terms from an employer's meetings. That cannot go through a tier whose terms allow training on it.

WARNING

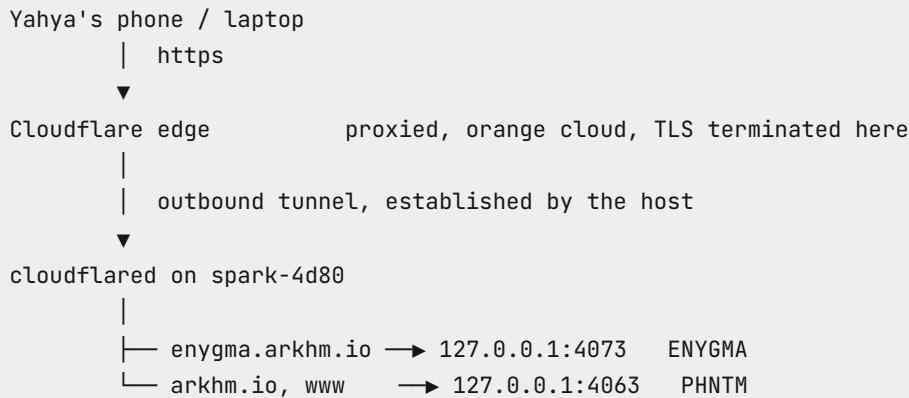
Do not run a real meeting through a project that is still on the free tier. Confirm the project reads as paid *before* the first upload, not after.

The key lives in **its own Google Cloud project**, not PHNTM's. The project is the unit of attribution and revocation: a second key inside PHNTM's project shares quota, billing lines and blast radius, and killing the project kills both systems. Restrict the key to the Gemini API.

A domain

`arkhm.io`, already owned. ENYGMA occupies the `enygma` subdomain and nothing else.

Network topology



The host opens no inbound port. `cloudflared` makes an outbound connection and the edge routes back down it, so there is nothing to port forward and nothing listening on the public internet. Both applications bind to loopback only.

Cost summary

LINE	COST	NOTE
Cloudflare	\$0	Free plan covers tunnel, DNS and TLS.
Domain	already owned	Shared with PHNTM.
Gemini API	cents per meeting	Audio is inexpensive; a \$10 prepay balance covers months at this volume.
Hardware	\$0 marginal	Shares <code>spark-4d80</code> with PHNTM.
Electricity	negligible	The machine was already running.

Prepay rather than postpay is the recommendation for a system handed to someone else: it fails closed. A runaway loop stops at the credit balance instead of arriving as a bill.

Build

Getting a release onto the host, configuring it, and proving it works before trusting it.

-
- 07 Deployment
 - 08 Configuration
 - 09 First run and verification

Chapter 7 | Deployment

PART 3 • BUILD

07

One script does the whole thing and is safe to run twice. It refuses to proceed rather than guessing, and it runs the test suite before it restarts anything.

Step 1 — Get the release onto the host

From a clone, which is the recommended path:

```
$ git clone https://github.com/dshafique/ENYGMA.git ~/enygma-release
$ cd ~/enygma-release
$ python3 tools/stamp_release.py
Mk II.23    build 23.2026.09.02.0134
```

`stamp_release.py` writes `VERSION` and `MARK` from the commit count. Both are build artifacts and are not in the repository, because committing them would mean the number changes the number it reports.

WARNING

If you deploy from an extracted archive instead of a clone, extract into a **clean** directory. Unpacking over an old copy leaves deleted files behind, and a stale template shadows a new one.

Step 2 — Run the installer

```
$ sudo bash deploy/install-on-spark.sh
```

In order, it:

1. **Announces the release** it is about to install and the one it is replacing. A stale deployment is visible before anything is touched.
2. **Checks Python** is 3.10 or newer, and that `venv` is available.
3. **Checks the port**. If something answers on 4073 and it is not ENYGMA, it stops. Two applications on one port is exactly the coupling this design avoids.
4. **Creates the unix user** if absent, and sets the permissions that make isolation real.

5. **Replaces release-owned directories** — `src/` `tools/` `tests/` `deploy/` — outright, then unpacks. `data/`, `uploads/`, `.env` and `.venv/` are the operator's and are never touched.
6. **Builds the virtualenv** and installs `requirements.txt`.
7. **Writes `.env` if absent**, with a freshly generated session secret at mode 600. An existing one is left alone.
8. **Proves it before wiring it up**: runs migrations, the token lint, the `.env` check and the full test suite *as the `enygma` user*. A failure here stops the deployment with the old version still running.
9. **Installs the systemd unit** and restarts.
10. **Verifies** `/healthz` answers and that no other account can read the database, the audio or the secret.

Step 3 — Backups, once per machine

```
$ sudo bash deploy/install-backups.sh
```

Takes one immediately so the first backup is not scheduled for tonight, verifies it by restoring it, installs `/etc/cron.daily/enygma-backup`, then runs the scheduled job directly to prove the schedule itself works and not merely the command.

What a correct install looks like

```
⇒ release 7.2026.08.27.0553 (from /home/arkhm/enygma-release)
   replacing build 6.2026.08.27.0546
   python3 3.12
⇒ code into /home/enygma/app
⇒ migrations and tests, as enygma
   applied 007_... (none pending)
ok: 52 tokens defined, every var() reference resolves
No problems found.
83 passed in 4.9s
⇒ systemd
   8589934592 memory ceiling in force
⇒ checks
   healthz: {"ok":true,...,"mark":"Mk II.23"}
   no other account can read the database, the audio or the secret
```

If the token count, the test count or the mark differ from what you expect, you are installing something other than what you think you are.

Chapter 8 | Configuration

PART 3 • BUILD

08

Everything comes from the environment. Nothing is hardcoded, and `.env` is data — it is parsed, never executed.

Editing it safely

```
$ sudo -u enygma nano /home/enygma/app/.env
```

WARNING

Paste that line **on its own**. An editor takes over the terminal, so anything pasted after it goes into the file instead of the shell.

Checking it before restarting

```
$ sudo -u enygma bash -c "cd /home/enygma/app && .venv/bin/python tools/check_env.py"
/home/enygma/app/.env mode 600
```

```
ENYGMA_PORT           4073
ENYGMA_PIPELINE        gemini
ENYGMA_SESSION_SECRET set, 64 chars, ends x7Qa
ENYGMA_GEMINI_API_KEY  set, 39 chars, ends 9fLk
```

```
No problems found.
```

Secrets are never printed, only described. The check reports stray non-assignment lines, duplicate keys, a `gemini` pipeline with no key, an empty session secret, and the wrong file mode. The installer runs it on every deployment.

SCAR

`run.sh` used to `source .env`, which means every line in it executed as the application user at each start. A pasted command block once landed inside that file. Under `systemd` it would have run on every restart, failed without a TTY, tripped `set -e` and produced a restart loop. It is now parsed as `KEY=VALUE` and anything else is named on `stderr` and ignored.

Switching on the real pipeline

```
ENYGMA_PIPELINE=gemini
ENYGMA_GEMINI_API_KEY=AIza...
```

```
$ sudo systemctl restart enygma
```

Confirm in **Settings**: the Pipeline card must read `GEMINI`, not `STUB`, and the amber banner on Meetings must be gone. The file saying `gemini` is not proof the process read it.

The full variable reference is Appendix A.

Chapter 9 | First run and verification

PART 3 • BUILD

09

Is it alive?

```
$ systemctl is-active enygma
active
$ curl -s https://enygma.arkhm.io/healthz
{"ok":true,"app":"enygma","port":4073,"version":"23.2026.09.02.0134","mark":"Mk
II.23"}
```

`/healthz` is the only route that needs no session. The `mark` it returns is the one thing to check after any deployment: if it is not the build you just installed, nothing else you observe means what you think it means.

Enrolling the first device

Open `https://enygma.arkhm.io`. With nothing enrolled the lock screen offers **Create your passkey** and nothing else — the first device to enrol becomes the owner.

NOTE

Enrol on the device that will be used most, usually the phone. Adding the second device afterwards is a two-minute job (Chapter 12); there is no penalty for choosing either order, but there is a penalty for enrolling on a machine you rarely open.

Setting the PIN

Settings → **PIN**. Four to ten digits. This is the way back in when the passkey is unavailable, and it is what bootstraps a second device if a setup code has expired. Set it before you need it.

A demo week, optional

```
$ sudo -u enygma bash -c "cd /home/enygma/app && .venv/bin/python tools/seed_demo.py"
wrote 5 demo recording(s). Remove with --clear.
```

Five meetings with transcripts, speakers, summaries and actions, so every tab has something in it while you look around. Everything it writes is tagged `source='demo'` and comes out again with `--clear`, which leaves real uploads untouched.

It seeds no headlines and no observations about the operator. Those would be inventions about a real person, and Tenet 05 applies to demo data too.

The first real recording

Drop a file on Meetings and watch it:

```
$ journalctl -u enygma -f
```

Expect `queued` for up to three seconds, then `transcribing`, then `ready`. The page refreshes itself while something is running — but only for what is actually on view, so reading one meeting is not interrupted because a different one is still working.

If it fails, the reason is on the meeting page verbatim, with a **Try again** that requeues without re-uploading. Chapter 17 lists what the reasons mean.

Access & Auth

How it reaches the internet, how it decides you are you, and what to do when that goes wrong. The half that is easiest to get subtly wrong.

-
- 10 Cloudflare Tunnel and DNS
 - 11 Passkeys and the freshness window
 - 12 Setup codes, PIN and adding a device
 - 13 Recovery procedures and threat model
 - 14 Installing as an app

Chapter 10 | Cloudflare Tunnel and DNS

PART 4 • ACCESS & AUTH

10

Mental model

`cloudflared` runs on the host and makes an *outbound* connection to Cloudflare's edge. Requests for `enygma.arkhm.io` arrive at the edge and are routed back down that connection. The host opens no port; there is nothing to forward and nothing exposed.

One `cloudflared` instance serves both applications. Its config maps hostnames to local ports, and ENYGMA's rule must sit **above the catch-all**, because the first match wins.

The two pieces

1. An ingress rule in the cloudflared config:

```
ingress:
  - hostname: enygma.arkhm.io
    service: http://127.0.0.1:4073
  - hostname: arkhm.io
    service: http://127.0.0.1:4063
  - service: http_status:404          # catch-all, always last
```

2. A DNS record in the zone:

TYPE	NAME	TARGET	PROXY
CNAME	enygma	<tunnel-uuid>.cfargotunnel.com	Proxied

WARNING

The orange cloud is not optional. An unproxied CNAME to `cfargotunnel.com` does not resolve at all — the name only exists inside Cloudflare's network. A grey cloud here produces *Server Not Found*, which reads like a DNS outage.

`deploy/dns-record.sh` reads the tunnel config and prints exactly the record required. It is read-only and changes nothing.

WARNING

The cloudflared config is serving PHNTM right now. Back it up before editing, validate after, and restore on any doubt. `deploy/add-tunnel-route.sh` does all three.

SCAR

`cloudflared tunnel ingress validate --config X` puts the flag in the wrong place; the flag must precede the subcommand. Worse, `cloudflared` exits 0 on that usage error, so a script that trusted the exit code printed *ingress rules are valid* having validated nothing at all. The script now reads the command's output, and restores the backup if it cannot find an explicit success.

Verifying, from outside

```
$ curl -s https://enygma.arkhm.io/healthz
{"ok":true,...,"mark":"Mk II.23"}
$ curl -s -o /dev/null -w "%{http_code}\n" https://arkhm.io/
200
```

Always check PHNTM in the same breath. The whole risk of touching this config is that ENYGMA comes up and something else quietly goes down.

Chapter 11 | Passkeys and the freshness window

PART 4 • ACCESS & AUTH

11

Mental model

A passkey is a private key generated by the device and held in its secure hardware, released only by a biometric or device unlock. The server stores the matching *public* key and a signature counter, and nothing else. There is no shared secret to steal.

The credential is bound to the origin that created it — `enygma.arkhm.io`, exactly. This is why `ENYGMA_RP_ID` and `ENYGMA_ORIGIN` must match what the browser sees, and why a passkey made against one hostname will not work against another.

Two levels of authorisation

Reading and writing are different questions, and ENYGMA answers them differently.

LEVEL	NEEDS	COVERS
View	A valid session, any age	Every page, the audio, the Markdown copy, term lookups. Anything the page is already showing.
Write	A session verified within the freshness window	Uploads, ticking actions, naming speakers, sending chat, minting a setup code, changing the PIN.

The window is `ENYGMA_REAUTH_MINUTES`, default 30. Going stale does not eject you: you keep reading, and the first write raises a small sheet asking you to confirm. Prove it is you and the original action is retried automatically — the page, the scroll position and anything typed all survive.

SCAR

Before that sheet existed the window was five minutes, and a stale upload returned `401` which the interface parsed as an empty result list and rendered as nothing at all. The file appeared to vanish. A silent no is the worst possible answer to *did that work*; every write now goes through one wrapper that either succeeds, re-authenticates and retries, or says what went wrong.

Lockout

Five failures across passkey, PIN and setup code *combined* — not five each — then a delay that doubles each further round, capped at an hour.

FAILURES	DELAY
5	30 seconds
10	60 seconds
20	4 minutes
50+	1 hour, capped

A flat delay is the wrong shape for a short PIN. A fat finger still costs thirty seconds; an exhaustive search of a four-digit PIN now costs more than a month of continuous attempts. A successful authentication clears the counter entirely.

The lockout is server-side and keyed on the account. A client-side countdown is decoration.

Chapter 12 | Setup codes, PIN and adding a device

PART 4 • ACCESS & AUTH

12

A passkey belongs to the device that made it. A second device has no way in on its own, and the two ways to give it one are not interchangeable.

The two credentials, which are not the same thing

	PIN	SETUP CODE
Looks like	0125 — 4 to 10 digits	B84C-5L28 — 8 letters and numbers
Lifetime	Standing, until changed	10 minutes, single use
Set from	Settings → PIN	Settings → Set up another device
Buys	A session	A session, in order to enrol one passkey
Use it when	The passkey is unavailable on a device already enrolled	A new device needs its own passkey

NOTE

Typing the PIN into the setup-code field is the single most likely mistake here, and the screen now names it as such rather than saying *not valid or has expired* — which was true, and useless, for every possible error.

Adding a device, the recommended way

1. On a device already signed in: **Settings → Set up another device**. An eight-character code appears. It is shown once and only its hash is stored.
2. On the new device: open `enygma.arkhm.io` → **Set up this device** → enter the code.
3. It signs that device in and immediately prompts to create a passkey there. Done.

Minting a code needs a *fresh* session, because it is the one credential that creates other credentials and a left-open tab should not be able to produce one. Creating a new code cancels any outstanding one: two live codes means one of them is forgotten, and a forgotten credential is the one that gets used against you.

Adding a device with the PIN instead

Open the site on the new device, choose **Use PIN instead**, then **Settings** → **Add this device**. This works and is the fallback when no signed-in device is to hand — but it means the standing secret gets typed on one more device, which is precisely what setup codes exist to avoid.

Removing a device

Settings → **Devices** → **Revoke**. The credential is marked revoked rather than deleted, so the record of what existed survives. A revoked passkey stops working immediately.

Chapter 13 | Recovery procedures and threat model

PART 4 • ACCESS & AUTH

13

Action: a device is lost or stolen

From any signed-in device: **Settings** → **Devices** → **Revoke**. Immediate.

If no other device is enrolled, sign in with the PIN first, then revoke, then enrol the replacement. If the PIN is also unknown, use the shell procedure below.

Action: locked out entirely

Shell access to the host is the ultimate authority. Nothing in the web interface can prevent an operator with root from getting back in.

```
$ sudo -u enygma bash -c "cd /home/enygma/app && .venv/bin/python -c \
    \"import sys; sys.path.insert(0, '.'); from src.auth import secrets_store as s;
    s.set_pin('482913')\""
```

Then sign in with that PIN and re-enrol. Change it afterwards from Settings — a PIN that has been typed into a shell has been written to history.

Action: rotate the session signing secret

Signs every session out, everywhere, immediately. Passkeys and the PIN are unaffected; only the cookies are invalidated.

```
$ sudo -u enygma nano /home/enygma/app/.env      # replace ENYGMA_SESSION_SECRET
$ sudo systemctl restart enygma
```

Generate one with `python3 -c "import secrets; print(secrets.token_urlsafe(48))"`.

Action: full auth reset

WARNING

This removes every passkey, the PIN and every outstanding setup code. The next visitor to the site becomes the owner. Do it only with the site unreachable, or immediately before enrolling.

```
$ sudo systemctl stop enygma
$ sudo -u enygma sqlite3 /home/enygma/app/data/enygma.db \
  "DELETE FROM credentials; DELETE FROM pin; DELETE FROM pairing_codes; DELETE FROM
  auth_attempts;"
$ sudo systemctl start enygma
```

Action: move to a new machine

1. Take a backup on the old host and copy it off.
2. Install on the new host, then stop the service.
3. Restore the database over the fresh one (Chapter 16).
4. Carry the `.env` across, or accept that a new session secret signs everyone out once.
5. Repoint the tunnel and DNS so the origin is unchanged.

WARNING

Passkeys are bound to the origin, not the machine. Keep `enygma.arkhm.io` and every enrolled device still works. Change the hostname and every passkey is dead — plan the PIN before, not after.

Threat model

THREAT	MITIGATION	RESIDUAL
Password reuse, phishing	No password exists	None
Credential stuffing	Public keys only; nothing to replay	None
PIN brute force	Argon2id, shared counter, doubling lockout	Low. A 4-digit PIN costs more than a month of continuous attempts.
Stolen unlocked device	30-minute freshness window on writes	Reading is open for the cookie's life. Revoke the device.
Another account on the host	Mode 700 data, 750 home, own unix user	Root can read everything. Root is the operator.
Cookie scoped to a sibling site	<code>__Host-</code> prefix forbids a Domain attribute	None. It is structurally impossible.
Transcript content at the model	Own project, paid tier, no training on paid data	Google processes the audio. This is the one unavoidable disclosure.
Transcript cached on a device	Service worker caches versioned static assets only	None. No HTML or API response is ever stored.
Disk failure	Nightly verified backups	Real. Backups are on the same disk. See Chapter 16.

Chapter 14 | Installing as an app

ENYGMA is a progressive web app. Installed, it gets a launcher icon, its own window with no browser chrome, and the phone's own share and multitasking treatment.

Android, Chrome or Samsung Internet

1. Open <https://enygma.arkhm.io> and sign in.
2. Menu → **Add to Home screen** (or *Install app*).
3. Confirm. The cipher grid appears in the launcher.

On a Fold, the installed app follows the fold: the tab bar layout on the cover screen, and more room on the inner display, with the nav rail appearing once the window is wide enough.

iOS, Safari

1. Open the site in **Safari**. Other iOS browsers cannot install.
2. Share → **Add to Home Screen**.

What changes when installed

ASPECT	BROWSER TAB	INSTALLED
Chrome	Address bar, tabs	None. Full height.
Launcher	A bookmark	An icon among the apps
Shortcuts	—	Long-press gives Meetings, Actions, Chat
Passkeys	Work	Work identically. Same origin, same credential.
Theme colour	—	Status bar follows light or dark

What the service worker does, and does not

It caches exactly one class of thing: requests to `/static/` carrying a `?v=` build stamp, whose contents can never change. That is what makes the app open instantly.

It never caches a page, an API response, audio, or anything under `/auth`. Everything ENYGMA renders is private; a cached page is a transcript sitting on the device's disk outside the file permissions that are the entire isolation story, and still servable after a sign-out. Offline, the app does not work — which is correct, because the data lives on a machine at home and pretending otherwise would mean keeping a copy that should not exist.

Updating after a deployment

Nothing to do. Every asset URL carries the build, so a new release is a new set of URLs and the old cache is discarded on activation. Closing and reopening the app is enough; there is no cache to clear by hand.

SCAR

Before assets were addressed by build, a browser served a stylesheet cached days earlier against new markup. The old file had no rule for the icons, and an inline SVG with no width fills its container — so a 20-pixel icon rendered as a 750-pixel chevron across the page. Icons now carry their own `width` and `height` as attributes, so CSS is an enhancement rather than load-bearing.

Operating

Using it, protecting it, reading its failures, and knowing what has and has not been proven.

15	Daily use
16	Backups and restore
17	Failure modes
18	Releases, and what is still unproven

Chapter 15 | Daily use

PART 5 • OPERATING

15

After a meeting

1. Open ENYGMA, go to **Meetings**, drop the audio file on the dropzone. Several at once is fine.
2. If you have been idle a while a sheet asks you to confirm it is you. Do so; the upload continues by itself.
3. Watch it move `queued` → `transcribing` → `ready`. The page refreshes itself while something on view is running.
4. Open it. Put real names on the speaker chips — this is the one thing the system cannot do for you, and it is what makes the Directory and the action owners meaningful.

Reading a meeting

Summary is the abstract, the decisions and the open questions. Every timestamp is a button: it seeks the audio to that moment, so any claim can be checked against what was actually said in about two seconds.

Transcript is the full record, with the current segment highlighted as the audio plays.

Actions is what was committed to; ticking one here ticks it everywhere.

Looking a word up

Press and hold any word in the transcript for about half a second. A small panel gives the definition. *Learn more* opens a Chat thread named for that term with the question already asked, and the conversation carries on from there.

This is the feature the system was built around. The words nobody stops to define in a meeting are the highest-value thing in the recording for someone still learning the domain.

Recording a meeting

Meetings has a record button above the drop zone. Press it, put the phone down, press stop. What comes back lands as a meeting like any other and goes straight into the pipeline. *Pause* holds it without ending it; *Discard* throws it away and asks first.

There are two recorders behind that one button and the page picks between them when it loads. In a browser it is the browser's own recorder, which works and runs **only while the page is open** — the screen going off or Android reclaiming the tab ends the recording, so leaving the page warns you first. Inside the native shell (Chapter 23) it is a foreground service with a notification, which Android will not kill, and the phone can go in a pocket for an hour.

NOTE

A browser records Opus inside a WebM container and gives no say in the matter, which is a format the transcription model does not read. ENYGMA remuxes it to Ogg on the way in — the same audio in a different box, not a re-encode, and about a second for an hour of speech. Without `ffmpeg` on the host the file is kept as it is rather than thrown away: better a recording that needs a retry than a meeting that is gone.

Asking, and stopping

An answer in Chat arrives as it is written rather than all at once when it is finished. On the local model that is the difference between a still screen for thirty seconds and a first sentence in two.

While it writes, the send button is a stop button. Press it and the answer stops where it is, and *what had already arrived is kept* — you pressed it because you had what you needed, so throwing the answer away at that moment would make the button pointless. The turn is marked *You stopped this*, and the mark is not decoration: the next question in that thread is answered with the whole conversation behind it, so the model is told where it was cut off rather than reading half a sentence as a finished thought and carrying on from a claim nobody completed.

NOTE

Stopping is a message to the server, not a dropped connection, so the server keeps and stores what it actually generated. It never takes the browser's word for what the model said. Local and Opus stream token by token; Gemini answers through its SDK in one piece, so there Stop throws the answer away rather than shortening it.

Stop is checked between chunks, so it lands within a word or two rather than instantly. On Opus, tokens already generated are still billed — Anthropic produced them whether or not you read them, and a stopped answer showing up as free would make the running total on Home wrong in the direction that matters.

Getting something out

Copy as Markdown on any meeting puts the whole thing on the clipboard, timestamps intact, ready to paste anywhere. There is deliberately no synchronisation to another system: two homes for one summary means neither is trusted, and the half that makes ENYGMA worth having — the claim welded to the moment, with the audio a click away — is the half that a pasted page throws out.

Weekly, thirty seconds

- **Actions** — anything still open that should not be.
- **Directory** — any speaker still unnamed. The count is in the subtitle.
- `journalctl -t enygma-backup -n 5` — backups still running.

Chapter 16 | Backups and restore

PART 5 • OPERATING

16

The database is the only thing here that cannot be reproduced. Audio can be uploaded again; a transcript, a summary, a corrected speaker name and an enrolled passkey cannot.

What is backed up, and what is not

ITEM	BACKED UP	WHY
<code>data/enygma.db</code>	Yes, nightly	Everything the system has ever understood.
<code>uploads/*</code>	No	Large, and re-uploadable from wherever it came from. Copying 500 MB files nightly onto the same disk buys very little.
<code>.env</code>	No	Two values, one of which should be rotated on any restore anyway.

Why not `cp`

The database runs in WAL mode. A file copy taken while a write is in flight is a torn database: it copies cleanly, it looks fine, and it fails on the day you need it.

`tools/backup.py` uses SQLite's own backup API, which produces a consistent snapshot even mid-write.

Every backup is verified when taken

A backup nobody has restored is a hope, not a backup. Each one is decompressed to a temporary file, opened, `PRAGMA integrity_check` run, and every expected table counted — immediately, as part of taking it.

```
$ sudo -u enygma sh -c "cd /home/enygma/app && .venv/bin/python tools/backup.py"
wrote enygma-20260827-045126.db.gz (8 KB)
restored enygma-20260827-045126.db.gz
  integrity_check: ok
  recordings      5
  transcript_segments 27
  credentials     1
the backup opens, passes integrity check and has every table.
```

Restoring

```
$ sudo systemctl stop enygma
$ sudo -u enygma sh -c "cd /home/enygma/app && \
  gunzip -c backups/enygma-20260827-045126.db.gz > data/enygma.db"
$ sudo systemctl start enygma
```

This procedure has been exercised, not assumed: a live database was deliberately overwritten with garbage, SQLite confirmed it unreadable, and the backup restored every recording, segment and action intact.

WARNING

The backups sit on the same NVMe as the database. That covers corruption, a bad migration and an accidental delete. It does **not** cover losing the disk. If this data matters beyond that, copy `/home/enygma/app/backups` somewhere else on a schedule. The system does not pretend to have solved this.

Retention

Fourteen nights by default; older ones are pruned as each new backup is taken. Pass `--keep 30` to `deploy/install-backups.sh` to change it.

Chapter 17 | Failure modes

PART 5 • OPERATING

17

What the system can tell you, and what each thing means. Every failure here has actually happened.

On a meeting

REASON SHOWN	MEANS	DO
<code>RuntimeError: The Gemini SDK is not installed...</code>	The virtualenv is missing <code>google-gemai</code> .	Re-run the installer, which installs <code>requirements.txt</code> .
<code>RuntimeError: ENYGMA_GEMINI_API_KEY is not set</code>	Pipeline is <code>gemini</code> with no key.	<code>check_env.py</code> , then set the key and restart.
<code>JSONDecodeError</code>	The model's response was malformed <i>and</i> nothing could be salvaged from it.	Run it again. If it repeats, the audio may be unusually long — see the note below.
<code>Gemini returned nothing that could be read...</code>	The model replied with prose, usually a refusal. The first 180 characters are quoted so you can see what it said.	Read the quote. Often the audio is silent or unreadable.
429 in the reason	Rate limited, or out of credit.	Check the billing balance. Retry.
<i>Incomplete</i> banner, still readable	The response was cut off; what arrived was recovered. The note says how far it reached.	Run it again for the whole thing, or keep what you have.
Stuck on <code>transcribing</code> after a restart	Should not happen: interrupted recordings are requeued at startup.	<code>sudo systemctl restart enygma</code> and watch the log.

On the way in

SYMPTOM	MEANS	DO
<i>Server Not Found</i>	DNS. Either the record is missing, or it is not proxied, or the browser cached an old NXDOMAIN.	<code>deploy/dns-record.sh</code> . In Firefox, <code>about:networking#dns</code> → Clear DNS Cache.
Cloudflare 502 or 1033	The tunnel is up but nothing is listening on 4073.	<code>systemctl status enygma</code> .
Passkey prompt never appears	Origin mismatch: the browser's origin is not <code>ENYGMA_ORIGIN</code> .	Check both values in <code>check_env.py</code> . They must match exactly, scheme included.
A write does nothing	Should not happen: a stale session raises the sheet.	Reload. If it persists, the browser has a stale <code>app.js</code> — but assets are build-addressed, so this means the deployment did not land.
<i>Locked for N seconds</i>	Five failed attempts across all methods.	Wait. The delay doubles each round; a successful sign-in clears it.
<i>That setup code is not valid or has expired</i>	Wrong code, used code, or more than ten minutes old. A PIN typed here is named as such.	Mint a fresh one from a signed-in device.

On the host

```
$ systemctl status enygma
$ journalctl -u enygma -n 100 --no-pager
$ journalctl -t enygma-backup -n 20
$ journalctl -t enygma-disk -n 20
$ curl -s https://enygma.arkhm.io/healthz
```

NOTE

`run.sh`: ignoring non-assignment line in `.env` in the log means exactly what it says: something in `.env` is not `KEY=VALUE` and was skipped. If it was a command you meant to run, it never ran. Fix it with `check_env.py` .

Long meetings, and the thirty minute wall

The platform supports speaker attribution for **up to thirty minutes of audio**. Beyond that it does not error: it stops distinguishing voices and keeps emitting whichever label it

used last. A fifty minute meeting comes back correctly attributed for half an hour and then as one uninterrupted monologue, and nothing about the result says so.

ENYGMA therefore cuts anything longer into overlapping windows, transcribes each inside the limit, and stitches the results. The windows are `ENYGMA_CHUNK_MINUTES` long, twenty-five by default so as not to sit on the boundary, overlapping by `ENYGMA_CHUNK_OVERLAP_SECONDS`.

Each window is diarized independently, so its `SPEAKER 1` has nothing to do with the previous window's. The overlap is what makes them reconcilable: the same speech is transcribed twice, once at the end of one window and again at the start of the next, and matching that repeated speech gives a mapping between the two label sets. Matching is on words rather than timings, because the two windows disagree about what time it is. A voice that only appears in the later window gets a new number rather than a guessed one.

WARNING

This needs `ffmpeg` on the host. Without it a long recording is still transcribed, in one piece, and carries a note saying attribution past the limit is not trustworthy. `check_env.py` reports whether it is installed.

To see how attribution actually behaved on a given recording:

```
$ sudo -u enygma sh -c "cd /home/enygma/app && \
    .venv/bin/python tools/inspect_transcript.py 12"
window      segs  distinct  distribution
25-30 min   16      3  SPEAKER 1×5  SPEAKER 2×6  SPEAKER 3×5
30-35 min   16      1  SPEAKER 1×16  ← one voice only
```

The output ceiling is a separate limit on how long a single transcript can be. The salvage path recovers what arrived and the meeting says how far it reached; raise `ENYGMA_MAX_OUTPUT_TOKENS` if it happens routinely.

Chapter 18 | Releases, and what is still unproven

PART 5 • OPERATING

18

The mark

Mk II.N, where N is the commit count of the repository the build came from. It is in the top right of every screen, in Settings, and at `/healthz`. A manual and a running system can always be checked against each other in one glance.

`VERSION` also leads with N and is appended to every static asset URL, so a browser cannot serve a cached stylesheet against newer markup.

Shipping a change

```
$ cd ~/enygma-release && git pull
$ python3 tools/stamp_release.py
$ sudo bash deploy/install-on-spark.sh
```

The installer runs the token lint, the environment check and the full test suite as the application user before it restarts anything. A failure leaves the previous version running.

What is proven

- **Isolation.** Verified by attempting the read as another account.
- **Restore.** Verified by destroying a live database and bringing it back.
- **Re-authentication.** Verified by driving a genuinely stale session in a browser.
- **The worker's failure path.** Verified with a backend that throws.
- **Truncation salvage.** Verified against the exact response that failed.
- **Windowing long audio.** Verified with real ffmpeg cuts on a 52 minute file, and end to end against a model that collapses exactly the way the real one does.
- **The service worker's discretion.** Verified by reading the cache after loading a meeting.

What is not

WARNING

Speaker attribution has never been measured. Labels come from asking a model politely in a prompt, not from a diarization system. They are a claim, not a measurement. Nobody has checked them against a known-good reference. For an operator reading his own meetings this is a fixable annoyance; before a name from here is quoted to anybody else, it should be verified by ear.

- **Transcript accuracy** has not been compared against a reference transcript of the same audio.
- **Stitched speaker identity across windows** has been proven on synthetic audio end to end, but not yet measured on a real recording with real voices at a real join.
- **The Friday note has never fired on real data.** The scheduling, the gathering and the voice rules are tested; a real week of his real meetings has not yet produced one.
- **Document content has only been generated by the stub.** Every renderer is verified by opening its output back with the library that reads that format, and the voice rules are tested against a deliberately badly-behaved model. What a real model writes into that structure, on his real material, has not been read by anybody yet.
- **The voice rules have never met a real model under pressure.** How often the first attempt trips a rule, and therefore how often a document costs two calls instead of one, is unknown until it runs for a week.
- **Off-machine backup** does not exist. Disk failure is total loss.
- **The consent conversation** about recording an employer's meetings is a matter between the operator and the people in the room, and no amount of engineering settles it.

NOTE

This chapter should shrink with each edition. A manual that claims everything works is a manual nobody has tested.

What it writes

Three things now leave this app with his name on them: a weekly note, a document in any of six formats, and every answer in Chat. This part is what they are, and the rule all three are held to.

19	The Friday note
20	Documents
21	How ENYGMA writes
22	The native shell

Chapter 19 | The Friday note

PART 6 • WHAT IT WRITES

19

Once a week ENYGMA writes the thing you would otherwise put off: three to five bullets on what you got done, three to five on what you are doing next, plain enough to paste into an email to your manager without editing it.

Where it appears

On Home, in a card called **Your week**, spanning the full width. It is the one thing on that screen that leaves the building, so it is laid out to be read the way its recipient will read it rather than as a dashboard tile.

When it is written

Friday at 15:00 local, set by `ENYGMA_WEEKNOTE_HOUR`. The check rides on the existing worker thread and only runs when no transcription is waiting, so it can never delay the audio you are standing there watching.

It is deliberately not a clock alarm. The host can be asleep at three on a Friday, and an alarm firing into a sleeping machine writes nothing and says nothing. Instead the worker asks a question with no clock in it: is there a finished week without a note? A missed Friday becomes a note waiting for you on Sunday rather than a week that never got written.

What it is built from

- **Done:** everything between Monday 00:00 and the end of Friday — actions you moved to done, meeting summaries, notes you added, documents you put in the Library.
- **Next:** your open and pending actions. Only work you actually owe, so the note cannot promise your manager something you never agreed to.
- **Never:** anything you rejected. Turning something down is a decision, not an achievement, and the model is shown those explicitly as declined so it cannot list them as work.

A line under the note says what it drew on, in words, so you can check it before you send it.

The four controls

- **Copy** puts it on the clipboard as plain text, ready for an email body. The text comes from the server rather than from the page, so what you paste is what was stored.
- **Edit** lets you fix a line in place.
- **Write it again** regenerates it, and warns first if you have edited it.
- **Write it in your own words** opens a box you type into. See below.

NOTE

Your edit is stored beside the generated note, never on top of it. The automatic Friday write cannot replace a line you fixed by hand. Pressing *Write it again* does replace it, which is why that one asks.

Writing it yourself

Everything above builds the note out of what ENYGMA saw: meetings, closed actions, documents. Most of a hardware week is not any of those. It happens at a bench, and the app sees none of it.

So there is a second way in. **Write it in your own words** opens a box on the card. Type the week the way you would say it out loud, one thing per line, spelling and grammar as they come, and press **Lay it out**. What comes back is the note.

```
re-flowed the ESP32 board, the I2C bus is stable now  
swapped the EZ0 probe  
still stuck on the humidity drift
```

The first two land under *what I got done*, the third under *what I am working on next week*.

NOTE

The model is allowed to arrange your words. It is not allowed to add any. If it tries — a bullet with "leveraged" in it, a fact you did not give it — the whole attempt is thrown away and your own sentences are used instead, split up but otherwise untouched. Worse prose, and true. The same thing happens if ollama is stopped or a key is missing, so a box you typed into always produces a note.

It saves into the same slot as a hand edit, which is what makes it safe to press. What you wrote *replaces* what the app inferred rather than being merged into it, so a week you described yourself wins over a week ENYGMA guessed at. Nothing is destroyed: *Write it again* still puts the built version back. A section you said nothing about keeps what was already there.

The default model answers, which is Local unless you have changed it, so nothing about your week leaves the Spark.

A quiet week

A week with nothing in it produces no note of its own, rather than a full one about work that did not happen. The first invented line would cost you every note after it. The card is still there, and the box is still on it, because a week ENYGMA saw nothing of is exactly the week worth writing yourself.

What it is quietly building

Every week is kept. After a term you have a dated, factual record of what you actually did, which is the document you want in front of you at a review and the one nobody ever has.

Chapter 20 | Documents

PART 6 • WHAT IT WRITES

20

Ask for a file and you get a file. Markdown, HTML, Word, PowerPoint, Excel or PDF, built from the conversation you are already having.

Two ways to ask

- **In words.** "Make me a .md file", "put that in a spreadsheet", "turn this into a slide deck". ENYGMA works out the format from how you asked. A general request for "a file" or "a document" is answered in Markdown.
- **Save as.** Every ENYGMA answer carries a small control for when you already have something worth keeping.

An ordinary question never produces a download. "What is a retained message?" is answered, not filed.

Where the file goes

Into the thread, with a download, under **Files in this thread** above the writing bar. Its prose also goes into the Library, so a question next month can quote a document made today.

One document, six formats

Everything renders from a single structure — headings, paragraphs, bullets, numbered lists, tables, code and quotes — rather than from six separate generators, because six generators means six chances to produce something that renders in one format and not the others.

- A table becomes a real table in Word, Markdown, HTML and PDF, and a sheet of its own in Excel.
- A level 1 or 2 heading starts a new slide, so a document becomes a deck without the model having to think in slides.
- The HTML is self-contained and needs nothing from the internet, so it opens from a Downloads folder with no signal.

- The PDF is built directly rather than printed from a browser, and its text is selectable and searchable. It is a document, not a picture of one.

House style

Plain and unbranded by default, which is what you want for anything going to a manager or into an employer's repository. The ENYGMA look is available on request. The letterhead in `docs/letterhead/` is the house style at its fullest, as PDF and Word, light and dark.

CAREFUL

Every document is a model call, and a slide deck is a large one. At seven documents a week the cost is pennies and disappears next to transcription. It is worth knowing the number rather than assuming it.

What it refuses

A document the model returned nothing usable for is refused rather than shipped. A file that downloads and opens on nothing looks like the feature worked, which is worse than an error.

Chapter 21 | How ENYGMA writes

PART 6 • WHAT IT WRITES

21

You put your name on what comes out of this app, so nothing leaving it should announce that a machine wrote it. The rules live in `src/voice.py`, and the Friday note, every document and every Chat answer are held to the same one.

Three habits, three answers

Rules in a prompt get ignored under pressure. These are checked after generation, in code.

- **Vocabulary.** Words almost nobody says out loud: leverage, delve, streamline, robust, seamless, deep dive, key learnings. Caught by list.
- **Typography.** The em dash, the semicolon in casual prose, curly quotes. Nobody reaches for an em dash on a phone keyboard. Caught by rewriting.
- **Shape.** "Here is your report." "In conclusion." Every bullet opening with a bolded label and a colon. An essay frame around a note. This is the one a word list misses entirely: a document can contain no banned word at all and still be unmistakable.

Asked twice, then put right

The first attempt is checked. If anything is wrong the model is asked again and *told exactly what*, because a bare retry repeats the mistake with more confidence. The second attempt is kept only if it is actually better; second is not the same as better.

After that the typography is corrected mechanically but the words are left alone. Deleting a sentence changes what the document says, and a report that quietly loses a clause is worse than one that reads a little stiffly.

THE RULE THAT MATTERS MOST

Technical vocabulary is never touched. I²C, MOSFET, EZO, pull-up resistor, bus address and diarization are the substance of the writing, not decoration on it. A filter that flattens those has made the document worse while congratulating itself. Cut management vocabulary, never engineering.

Code is exempt

A fenced code block passes through byte for byte and is not read when deciding whether to ask again. So `i2cdetect -y 1; make robust-build` neither loses its semicolon nor sends the whole answer back for a rewrite. Table rows are data too and keep their punctuation.

Chapter 22 | The native shell

PART 6 • WHAT IT WRITES

22

An Android app that loads the live ENYGMA from the Spark. The web app stays the one source of truth and updates the moment you deploy; the shell exists for the one thing a browser tab cannot do.

Why bother

Recording. In a browser the recorder runs only while the page is open: the screen goes off, Android reclaims the tab, the meeting is gone. Inside the shell the recording runs in a foreground service with an ongoing notification, which Android will not kill, so the phone can go in a pocket for an hour.

Second, orientation. Android bakes an orientation into an installed package at install time, and a web manifest claiming `"orientation": "any"` is an explicit "I handle every orientation" that overrides the phone's rotation lock. In the shell that value is one line of `AndroidManifest.xml` you control, set to `unspecified`, which means *follow the phone*.

Where it lives

`native/capacitor/` in this repository, so the shell's configuration and the web app it loads cannot drift apart. Its `README.md` is the build procedure and is the file to read before touching any of this. The generated `android/` directory is not versioned: `npx cap add android` writes it in ten seconds from the config that is.

The two hand edits

`cap sync` cannot make either of these, and forgetting one is a bug that only appears on the phone.

- `android:foregroundServiceType="microphone"` on the service. With the wrong type, Android 14 and up refuses to start it and the recording quietly becomes an ordinary one that dies with the screen.
- `android:screenOrientation="unspecified"` on `MainActivity`, plus the `RECORD_AUDIO`, `FOREGROUND_SERVICE_MICROPHONE` and `POST_NOTIFICATIONS` permissions.

The Friday reminder

The note is written whether or not anyone looks at it, which is the problem: a note nobody opens is a note nobody sends. So inside the shell the phone schedules itself a reminder for Friday afternoon, an hour after `ENYGMA_WEEKNOTE_HOUR`, and taps him on the shoulder.

Local, not pushed. A push notification would mean a Firebase project, a device-token table and a sender in the worker, and would put a Google dependency in the middle of a system that is otherwise entirely his. "It is Friday afternoon" is a fact the phone already knows, so it does not need telling. The page reschedules it on every launch under one id, which replaces rather than stacks, so changing the setting moves the reminder and a hundred launches do not make a hundred alarms.

The two notifications a local alarm cannot give are the valuable ones: a transcription that finished, and a transcription that failed. The second matters most, because a failure currently sits silently until he happens to look. Both need push, and push needs Firebase.

What it is honest about

The shell runs on Capacitor's `server.url`, which Capacitor's own documentation calls "not intended for use in production". That warning is about app-store review and about losing offline behaviour. Neither binds here: this is sideloaded onto one phone, and ENYGMA has never worked offline because every page is rendered on the Spark. What it does mean is that when the Spark or the tunnel is down, the app shows a single sentence saying so rather than anything useful. That is the price of the web app being the one source of truth, and it is worth paying.

It also changes nothing about the interface. Same HTML, same CSS, same layout. A shell is a jacket, not a redesign.

Reference

Every knob, every route, the shape of the source tree, and the list of decisions that look strange until you know what caused them.

A	Environment variables
B	HTTP surface
C	Source tree map
D	The scar list

Appendix A | Environment variables

REFERENCE

A

Every value the application reads. Extracted from `src/config.py` at this edition.

VARIABLE	DEFAULT	MEANING
ENYGMA_PORT	4073	Loopback port. PHNTM holds 4063.
ENYGMA_HOST	127.0.0.1	Bind address. Loopback only; the tunnel is the way in.
ENYGMA_RP_ID	enygma.arkhm.io	WebAuthn relying party. Must equal the browser's hostname.
ENYGMA_RP_NAME	ENYGMA	What the passkey prompt calls this site.
ENYGMA_ORIGIN	https://enygma.arkhm.io	Exact origin, scheme included. A mismatch means no passkey prompt.
ENYGMA_SESSION_SECRET	<i>required</i>	Signs the session cookie. Its own, never PHNTM's. Changing it signs everyone out.
ENYGMA_REAUTH_MINUTES	30	How long a session stays fresh enough to write.
ENYGMA_INSECURE_COOKIES	0	Drops <code>__Host-</code> and <code>Secure</code> . Local http development only.
ENYGMA_TZ	<i>machine's zone</i>	Display timezone. Storage is always UTC.
ENYGMA_PIPELINE	stub	stub or gemini. Stub needs no key and no network.
ENYGMA_GEMINI_API_KEY	<i>required for gemini</i>	Its own key, in its own project, on the paid tier.
ENYGMA_GEMINI_MODEL	gemini-3.5-flash	Switching models is this line and a restart.
ENYGMA_MAX_OUTPUT_TOKENS	65536	Output ceiling. Too low truncates long transcripts.
ENYGMA_MAX_UPLOAD_MB	500	Per-file limit for audio, checked before a byte is written.
ENYGMA_WEEKNOTE_HOUR	15	Hour on Friday, local, when the week's note is written. Mid-afternoon rather than end of day, so it is read and edited before he leaves rather than found on Monday describing a week he has stopped thinking about.
ENYGMA_CHUNK_MINUTES	25	Window length for long audio. Must stay under the thirty minute diarization limit.

ENYGMA_CHUNK_OVERLAP_SECONDS	90	How much speech two windows share. This is what carries speaker identity across a join.
ENYGMA_MAX_UPLOAD_MB	500	Per-file limit, checked before a byte is written.
ENYGMA_WORKER_POLL_SECONDS	3	How long an idle worker waits before looking again.
ENYGMA_HINOTES_ENABLED	0	The recorder poller. Gated off; audio arrives by drag and drop.

NOTE

Modules are gated in configuration, never deleted. The HiNotes poller is switched off, not removed, so turning it back on is one line rather than an archaeology exercise.

Appendix B | HTTP surface

REFERENCE

B

43 routes. **View** needs a session of any age; **Write** needs one inside the freshness window; **Open** needs none.

METHOD	PATH	AUTH	PURPOSE
GET	/healthz	Open	Liveness, build and mark.
GET	/lock	Open	The lock screen.
GET	/sw.js , /manifest.webmanifest	Open	PWA. Must be at the root to control scope.
GET	/	View	Home.
GET	/meetings , /meetings/{id}	View	List and reader.
GET	/meetings/{id}/audio	View	Streams the recording.
GET	/meetings/{id}/markdown	View	The copy-out.
GET	/actions , /directory , /settings	View	The remaining tabs.
GET	/chat , /chat/{id}	View	Threads.
GET	/api/term?q=	View	Glossary lookup behind the popover.
GET	/api/meetings	View	The listing as JSON.
GET	/auth/devices	View	Enrolled passkeys.
POST	/upload	Write	One or many audio files.
POST	/meetings/{id}/retry	Write	Requeue without re-uploading.
POST	/meetings/{id}/speaker	Write	Put a name on a label.
POST	/actions/{id}/toggle	Write	Tick or untick.
POST	/chat/new , /chat/from-term , /chat/{id}/say	Write	Start a thread, or one turn of it.
POST	/chat/{id}/stream	Write	One turn, as server-sent events. What the page uses; /say is the no-JavaScript fallback.
POST	/chat/{id}/stop	Write	Stop the answer being written. Keeps what has arrived.
POST	/directory/{name}	Write	Role, org and note for a person.
POST	/auth/pin/set , /auth/pin/clear	Write	Manage the PIN.
POST	/auth/pairing/new	Write	Mint a setup code. Fresh session required.
POST	/auth/devices/{id}/revoke	Write	Revoke a passkey.

POST	<code>/auth/register/*</code> , <code>/auth/login/*</code>	Open*	WebAuthn. Registration is open only while nothing is enrolled.
POST	<code>/auth/pin/verify</code> , <code>/auth/pairing/redeem</code>	Open	Rate limited by the shared counter.
POST	<code>/auth/logout</code>	Open	Clears the cookie.
POST	<code>/chat/{id}/document</code>	Write	Save as: build a document from one answer, or from the thread.
GET	<code>/files/{id}</code>	View	Download a document ENYGMA made. Always an attachment, never inline.
POST	<code>/week/{monday}/again</code>	Write	Regenerate the Friday note. Replaces an edit, deliberately.
POST	<code>/week/{monday}/words</code>	Write	Lay out free text as the note. Saves into the edit slot.
POST	<code>/week/{monday}/edit</code>	Write	Store his version beside the generated one, never on top of it.
GET	<code>/week/{monday}/text</code>	View	The note as plain text, which is what Copy reads.

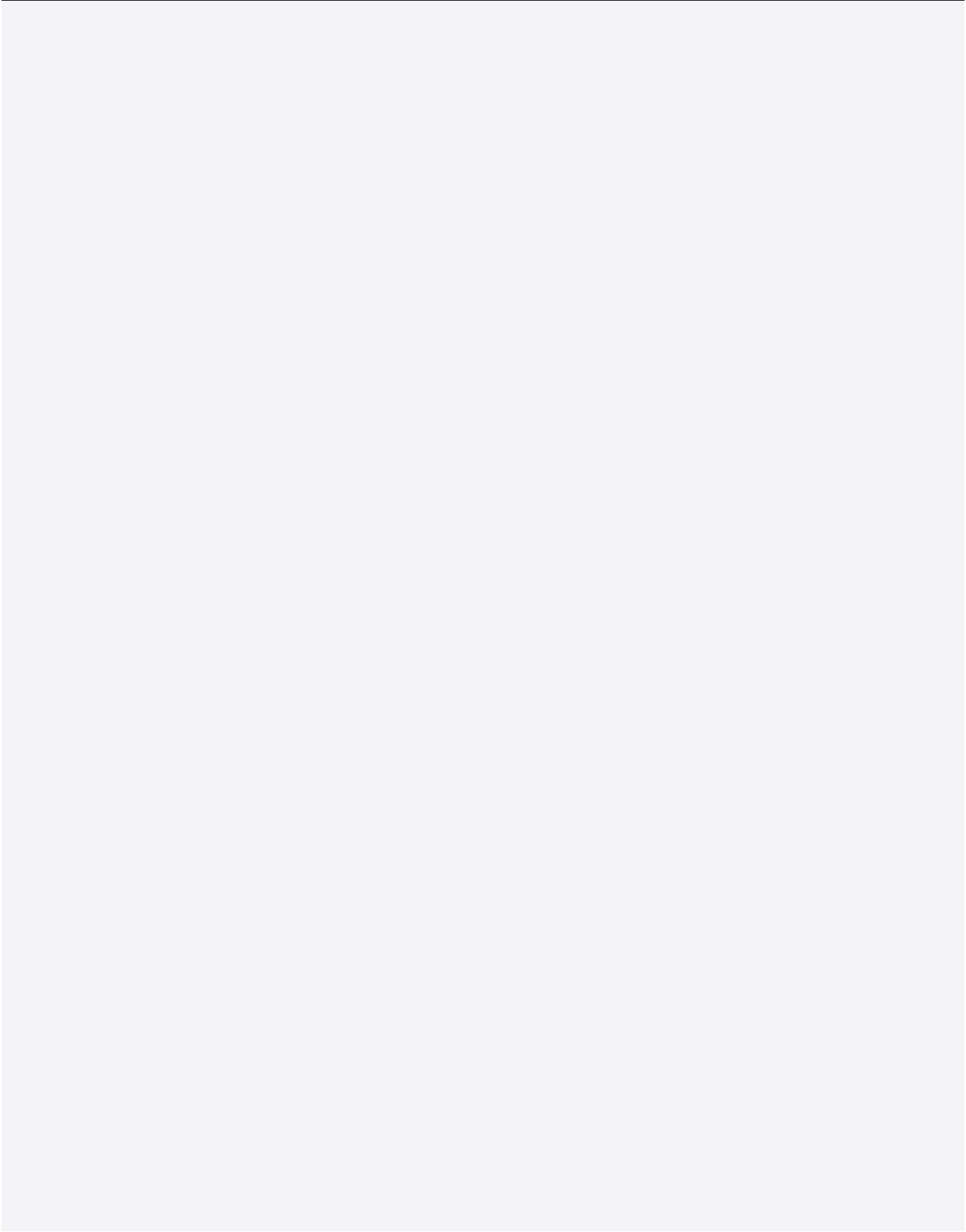
NOTE

A 401 carries a machine-readable reason: `locked` sends the browser to the lock screen, `stale` raises the re-auth sheet and retries. A request that says it accepts HTML gets an error page; anything else gets JSON, because a `fetch()` receiving HTML fails in a way that is far harder to read.

Appendix C | Source tree map

REFERENCE

C



```

enygma/
├─ run.sh                parses .env, never sources it; starts uvicorn
├─ requirements.txt      pinned, including google-genai
├─ VERSION MARK         build artifacts, not in the repository
├─
├─ src/
│   ├─ config.py         every value from the environment
│   ├─ db.py             connection and the migration runner
│   ├─ main.py           routes; require_view vs require_session
│   ├─ fmt.py            timestamps: stored UTC, shown local
│   ├─ meetings.py       the read layer for recordings
│   ├─ home.py actions.py directory.py chat.py glossary.py
│   ├─ library.py        documents, passages, retrieval
│   ├─ voice.py          one rule for every word that leaves the app
│   ├─ weeknote.py       the Friday note: gather, write, keep
│   ├─ documents.py      one structure, six renderers, no system deps
│   ├─ made.py           the files it built, on disk under data/
│   └─ export.py         one meeting as Markdown
│   └─
│       ├─ migrations/   forever-additive, applied in filename order
│       │   ├─ 001_init.sql recordings, operator, ingest log
│       │   ├─ 002_passkeys.sql credentials, challenges, pin, recovery
│       │   ├─ 003_uploads... segments, speakers, summaries, actions
│       │   ├─ 004_workspace.sql people, chat, terms, observations, tidbits
│       │   ├─ 005_pairing.sql setup codes
│       │   └─ 006_notes.sql a non-fatal remark on a recording
│       └─
│           ├─ auth/
│           │   ├─ passkeys.py WebAuthn registration and assertion
│           │   ├─ session.py  __Host- cookie and the freshness window
│           │   ├─ attempts.py one counter for every method, doubling delay
│           │   └─ secrets_store.py Argon2id: PIN, setup codes, recovery
│           └─
│               ├─ ingest/upload.py stream, hash and dedupe in one pass
│               └─ pipeline/
│                   ├─ base.py    Segment, Transcript, Summary
│                   ├─ stub.py    no key, no network, deterministic
│                   ├─ gemini.py  json mode, output ceiling, salvage parser
│                   └─ runner.py  the worker; requeues interrupted jobs
│               └─
│                   ├─ templates/ base is the shell; _meeting is the reader
│                   └─ static/
│                       ├─ css/tokens.css GENERATED. Never edited by hand.
│                       ├─ css/app.css   references only, no raw hex
│                       ├─ js/app.js     one module; every write goes through send()
│                       ├─ sw.js         versioned static assets only
│                       └─ icons/        generated from the mark specification

```

```

├─ tools/
│   ├── gen_tokens.py      the palette, and the only place hex appears
│   ├── check_tokens.py    fails the build on an undefined var()
│   ├── check_env.py       .env sanity, secrets masked
│   ├── backup.py          SQLite backup API, verified on every run
│   ├── seed_demo.py       a demo week; --clear removes it
│   ├── gen_icons.py       PWA icons from the mark
│   └─ stamp_release.py    VERSION and MARK from the commit count
├─ deploy/
│   ├── install-on-spark.sh idempotent; tests before it restarts
│   ├── install-backups.sh  nightly, and proves the schedule runs
│   ├── enygma.service      own user, MemoryMax, ProtectSystem=strict
│   ├── add-tunnel-route.sh  backs up and validates the cloudflared config
│   ├── dns-record.sh       read-only; prints the record required
│   └─ disk-ceiling.sh      quota if possible, hourly alarm otherwise
├─ tests/                  90 tests
└─ docs/manual/           this document

```

Appendix D | The scar list

REFERENCE

D

Every one of these looks like an odd decision until you know what caused it. They are collected here because this is the appendix that will save the next person the most time.

WHAT HAPPENED	WHAT IT NOW DOES
<code>adduser</code> left the home directory world-readable, so another account on the machine could read every recording and the database.	The installer <code>chmods</code> 750 and 700, then <i>proves</i> it by attempting the read as another account.
A semicolon inside an English sentence in a migration comment ended the statement early; the rest of the prose became SQL and failed as <code>near "the": syntax error</code> .	Comments are stripped before splitting on semicolons, respecting quoted strings.
CSS referenced custom properties that nothing defined. The declarations were silently dropped and a layout collapsed with nothing in the console.	<code>check_tokens.py</code> fails the build on any unresolved <code>var()</code> . The installer runs it.
A route rendered a template nobody had looked at in weeks, because every test called the page beneath it directly.	Every page is requested over HTTP in the suite, and a test asserts every template <code>main.py</code> names exists.
An install re-deployed a stale extraction and reported complete success. It was three days behind.	Every build carries a mark. The installer announces what it is installing and what it is replacing; <code>/healthz</code> reports it.
<code>cloudflared</code> exited 0 on a usage error, so a script reported "ingress rules are valid" having validated nothing.	The script reads the command's output, and restores its backup unless it finds explicit success.
A browser served a stylesheet cached days earlier against new markup. Icons had no width rule, and an inline SVG with no width fills its container — a 20px icon became a 750px chevron.	Every asset URL carries the build. Icons also carry <code>width</code> and <code>height</code> attributes, so CSS is an enhancement rather than load-bearing.
<code>run.sh</code> sourced <code>.env</code> , so every line executed as the application user. A pasted command block landed in that file.	<code>.env</code> is parsed as <code>KEY=VALUE</code> . Anything else is named on <code>stderr</code> and ignored.
A stale session made an upload return 401, which was parsed as an empty result list and rendered as nothing at all. The file appeared to vanish.	The re-auth sheet, and one wrapper around every write that either succeeds, re-authenticates and retries, or says what went wrong.
<code>google-genai</code> was never in <code>requirements.txt</code> . The real pipeline had never once been capable of running.	Pinned, with a test asserting it is declared, and an import error that names the command to fix it.
Inline audio was sent as raw bytes instead of base64, which fails at the transport, so the error would never have mentioned audio.	Base64, with every payload shape probed against the SDK's own request normaliser in a test.
SQLite stores UTC. The screen also showed UTC, so a meeting recorded at ten past midnight read 5:11 AM.	Stored UTC, rendered in the machine's zone, overridable with <code>ENYGMA_TZ</code> .

A long transcript came back cut off. Asking for one large JSON document meant a single bad character discarded the entire meeting.	Constrained JSON decoding, a raised output ceiling, and a salvage parser that keeps every segment that arrived.
A recording is marked <code>transcribing</code> before work starts. A crash in that gap left it there forever, showing a progress bar that could never finish.	<code>requeue_stuck()</code> at startup returns them to the queue with a note.
A long transcript was requested as one JSON document, so a response cut off at the end discarded the whole meeting.	The model is constrained by a JSON schema, the output ceiling is raised, and a salvage parser keeps every segment that arrived.
The fix for that sent <code>response_mime_type</code> without its required partner and produced a fresh 400, after the audio had already been uploaded.	Optional request keys are optional: a rejection about the request's shape falls back to the minimum request. A wrong guess costs one call, not a meeting.
Everything after the thirty minute mark of a meeting was attributed to one speaker. Nothing errored; the transcript simply became wrong halfway through and still looked complete.	Diarization is a documented thirty minute feature. Long audio is cut into overlapping windows, each diarized inside the limit, and speaker identity is carried across the joins by matching the speech the windows share.
A trigger body contains semicolons, and the migration runner split statements on them. The trigger arrived in pieces, each failing as "incomplete input" — an error that says nothing about triggers.	The splitter rejoins fragments while a <code>BEGIN</code> is still open.
Library search matched a question about swallows to a document about message brokers, because both contain "what" and "the".	Stopwords are removed, and a passage must share two distinctive words with the question or one that is rare across the library. Otherwise nothing is returned.
The setup-code field answered every possible mistake — a typo, an expired code, a used code, a PIN — with the same sentence.	A four-digit entry is named as a PIN; a wrong length says what the length should be.
Generated documents were stored in <code>app/made</code> , beside <code>data/</code> rather than inside it. The service unit runs <code>ProtectSystem=strict</code> and grants exactly two writable paths, so the directory was read-only at the kernel level whatever its permissions said. Every test passed; it failed on the host with <code>[Errno 30]</code> the first time he asked for a file.	Documents live under <code>data/made</code> . A test reads <code>ReadWritePaths</code> out of the unit file and scans the source for any store hung off <code>BASE_DIR</code> that is not one of them, because no runtime test in a temp directory could ever catch this.
The writing bar in Chat was an ordinary block at the end of a growing page. On a thread with four answers it sat 163 pixels	The composer is stuck to the bottom of the screen and the transcript scrolls behind it.

below the fold and reaching it meant scrolling the whole conversation. Reported as "the writing bar doesn't fit the phone".	Both folding-phone widths, 344 and 830, are measured on every change.
Hiding the tab bar while typing meant pressing Send blurred the box, which put the bar back, which moved the composer sixty pixels between his finger going down and coming up. The button slid out from under his thumb as he pressed it and nothing happened at all.	Send no longer takes focus off the box, and the bar returns only once focus has genuinely left the composer.
Enter sent the message. On a phone there is no Shift to hold, so the only key that starts a new line was the key that posted a half-finished sentence — four times, with nothing guarding a resend.	Enter sends only where a Shift key exists. The composer is disabled while a request is in flight and the field is cleared before it leaves.
The send button became the stop button, and the box is cleared the moment a message goes. So pressing Stop ran the browser's own validation on an empty <code>required</code> field, which refused the submit and showed "Please fill out this field". The handler never ran, the answer carried on writing itself, and the button did nothing. Every test passed; only a browser could find it.	The form is <code>novalidate</code> and emptiness is checked in the handler. A test asserts both, and the flow is driven in a real browser on every change.
Chat had no memory. Every turn was answered cold, so a follow-up had nothing to follow and the same question twice produced two unrelated essays.	The last twelve turns go into the prompt, and the same words inside twenty seconds return what was already said.
The installer ended by saying the pipeline was still the stub while its own environment report, four lines above, showed <code>gemini</code> and a key. The false half was the one written as an instruction.	The closing advice is read from the installed <code>.env</code> .
In the letterhead, tracking set for a mono label leaked into every plain string after it: <code>Tc</code> is PDF graphics state, not text-object state. The letter body came out letter-spaced and ran off the right margin, and the wrapper could not know because it measures without tracking.	Tracking is reset to zero inside the same text object. A test measures a rendered line.

END OF VOLUME I